

Самостійна робота №5: Server-Side Rendering та Next.js

Тема: Server-Side Rendering та Next.js

Мета: Зрозуміти різницю між SSR, SSG та CSR, ознайомитися з фреймворком Next.js, навчитися створювати проєкт та налаштовувати базову маршрутизацію.

Кількість годин: 4

Теоретичний матеріал

Стратегії рендерингу

CSR (Client-Side Rendering) - стандартний підхід React: сервер віддає порожній HTML, JavaScript завантажується та рендерить сторінку у браузері. Переваги: інтерактивність, менше навантаження на сервер. Недоліки: повільне First Contentful Paint (FCP), проблеми з SEO (пошукові роботи можуть не виконувати JS).

SSR (Server-Side Rendering) - HTML генерується на сервері для кожного запиту. Браузер отримує готовий HTML, після чого JavaScript "гідратує" сторінку, додаючи інтерактивність. Переваги: швидкий FCP, хороше SEO. Недоліки: навантаження на сервер, повільніша навігація між сторінками.

SSG (Static Site Generation) - HTML генерується на етапі збірки, а не на кожний запит. Готові HTML-файли розповсюджуються через CDN. Переваги: максимальна продуктивність, мінімальне навантаження на сервер. Недоліки: не підходить для часто оновлюваного контенту.

ISR (Incremental Static Regeneration) - гібрид SSG та SSR: сторінки генеруються статично, але регенеруються через заданий інтервал часу.

Next.js - огляд фреймворку

Next.js - React-фреймворк для продакшн-застосунків від Vercel. Підтримує всі стратегії рендерингу, має вбудовану маршрутизацію на основі файлової системи, API Routes, оптимізацію зображень та шрифтів, middleware.

App Router (Next.js 13+)

Сучасний підхід до маршрутизації у Next.js. Файли розміщуються у директорії `app/`. Кожна папка - маршрут, а файл `page.tsx` - сторінка. Спеціальні файли: `layout.tsx` (обгортка для сторінок), `loading.tsx` (стан завантаження), `error.tsx` (обробка помилок), `not-found.tsx` (404). Динамічні маршрути: `app/posts/[id]/page.tsx`.

Server Components та Client Components

Next.js App Router використовує React Server Components за замовчуванням. Серверні компоненти рендеряться на сервері, не включають JS у бандл клієнта, можуть безпосередньо звертатися до БД. Клієнтські компоненти позначаються директивою `'use client'` - потрібні для інтерактивності, хуків, обробників подій.

Контрольні питання

1. У чому різниця між CSR, SSR та SSG? Наведіть приклади застосунків для кожного підходу.
2. Що таке гідратація (hydration) і навіщо вона потрібна при SSR?
3. Як працює файлова маршрутизація в Next.js App Router?
4. Яка різниця між Server Components та Client Components? Коли використовувати кожен тип?
5. Що таке ISR і яку проблему він вирішує?
6. Як створити динамічний маршрут у Next.js (наприклад, для сторінки окремого поста)?
7. Для чого використовуються файли `layout.tsx`, `loading.tsx` та `error.tsx`?

Практичне завдання

1. Створіть проєкт Next.js: `npx create-next-app@latest my-blog --typescript --app`.
2. Створіть три сторінки: головна (`/`), про нас (`/about`), блог (`/blog`).

3. Додайте спільний `layout.tsx` з навігацією між сторінками.
4. Створіть динамічний маршрут для окремого поста (`/blog/[slug]`).
5. Додайте `loading.tsx` для сторінки блогу.

Порядок виконання

1. Вивчити теоретичний матеріал, наведений у цьому документі.
2. Ознайомитись з рекомендованими джерелами.
3. Відповісти на контрольні питання.
4. Виконати практичне завдання.

Рекомендовані джерела

- [Next.js - офіційна документація](#) - повна документація Next.js
- [Next.js App Router](#) - документація App Router
- [React Server Components - RFC](#) - специфікація React Server Components
- [Rendering on the Web - web.dev](#) - порівняння стратегій рендерингу
- [Next.js Learn](#) - інтерактивний курс від Vercel